Genesis：AI色認識でSO-101が検査ボタンを操作

Physical AI 応用1講座 最終課題 ／ 難易度：初級〜中級

OmniCampusアカウント名：shirakashi

1. 課題の概要と目的

実機を使わず、SO-101アームが色で指定された検査ボタンを順番に押す。指示は「赤 → 緑 → 青」。各ボタンに近づいて押し、先端を離して待機位置へ戻る。設備の簡単な起動・動作検査をロボットに任せる場面を想定した。設備の異常診断そのものは実装していない。

2. 使用したSO-101

TheRobotStudio/SO-ARM100が公開する公式SO-101 followerのURDFと13個のSTLを使用した。5軸アームとグリッパーの関節・可動範囲・形状は元の定義に従う。グリッパーを閉じたまま保持し、先端へ25mmの固定プローブと直径18mmの接触球を追加した。元モデルと変更版を分け、Apache-2.0ライセンス、出典、固定コミット、ファイルのハッシュを同梱した。

3. AIを使った部分

固定した俯瞰カメラで実際に描画された画像を取得する。既知のボタン位置に対応する6×6画素の平均RGBを明るさで正規化し、学習済みの多クラスロジスティック回帰で赤・緑・青を分類する。分類された色とボタン位置を対応づけ、色の指示からアームの押下対象を選ぶため、AI推論結果がロボットの動作につながる。

学習は明るさとノイズを変えた人工RGBデータ3,600件で行い、別の乱数種で作った900件で評価した。学習済み係数をJSONで同梱し、同じ条件で再学習も可能。AIの学習・推論はCPUで実行し、外部AI API・クラウド・実機は使用しない。

4. 制御と成功判定

講義で挙げられたGenesis 0.3.3を使用し、重力・接触・摩擦を含む物理計算をGPUで行う。可動範囲を守る逆運動学をNumPy/SciPyで解き、目標関節角を滑らかに補間してGenesisのPD制御へ送る。初期配置後、アームはモーターだけで動く。上下に動くボタンと先端の物理接触が発生し、4mm以上沈んだときに押下成功とする。映像もGenesisのカメラから取得する。

5. 実行結果

項目	実測結果
タスク	赤 → 緑 → 青の押下・待機位置へ復帰、成功
動画	72秒、1280×720、20fps、MP4/H.264
再生速度	1倍速（シミュレーション時間）
押し込み量	青/赤/緑：10.1/9.9/10.1 mm
先端以外のアームとボタン接触	0 件
カメラ認識	A＝青、B＝赤、C＝緑：3/3一致
人工データでの分類精度	100.0%（900件）



図：Genesis内のSO-101が赤ボタンを押す。接触でボタンが沈み、画面に完了数を表示。

6. 工夫した点と限界

SO-101の題材をベースに、カメラのAI認識、色の順序指定、接触で沈むボタン台を組み合わせた。成功表示は先端の接触と押し込み量に基づく。AIとアーム制御の役割を分け、複雑な学習型制御を避けた。別の指示「青 → 赤 → 緑」でも動作を確認した。

ボタン位置は既知で、固定画像領域を読むため物体検出は行っていない。学習・評価は人工データで、実環境の認識精度は未評価。追加プローブ、接触力、関節変位はシミュレーションのもので、実機の調整やカセンサーは検証していない。実機化にはカメラとアームの座標合わせ、接触力の制限、周囲の安全確認が必要となる。

7. 環境・参考・提出物

Windows 11 / WSL2 Ubuntu 22.04 / Python 3.10 / Genesis 0.3.3 / CUDA対応PyTorch / RTX A2000 12GB。NumPy・SciPyで逆運動学、scikit-learnで色分類、Pillow・imageioで動画、ReportLabでPDFを作成。

参考：最終課題（10/19改訂版）、Genesis公式資料、指定サンプル「SO-101入門」、TheRobotStudio/SO-ARM100公式モデル（Apache-2.0）。URLとモデルの変更内容はREADME.md・THIRD_PARTY.mdに記載。タスク用コードは独自実装。

提出物：本PDF、outputs/demo.mp4、コード一式、SO-101モデル、学習済み係数、依存ファイル、README.md。提出・再実行の方法は提出手順.mdに記載。